Conference Review Assignment Tool

Design of Algorithms - Programming Project I

Spring 2026 · L.EIC016

Team Members:
Pedro Tomás Teixeira - up202404987
Rafael Pinho e Silva - up202406334
Group: T09 G01

System Architecture

CSV Input → CSVParser → InputValidator → Data Models → FlowNetwork (Dinic's) → OutputWriter → CSV Output

Data Models

ConferenceData.h
AssignmentData.h

Algorithm

FlowNetwork.h/.cpp
Dinic's Max-Flow

Validation

InputValidator.h/.cpp
Field + Cross checks

I/O & UI

CSVParser,
OutputWriter
Menu,
DisplayFormatter

Graph.h - Professor-provided templated graph with Vertex/Edge/Flow + reverse edge support. NOT MODIFIED.
FlowNetwork.h has a MutablePriorityQueue forward declaration to work around Graph.h's friend class reference.

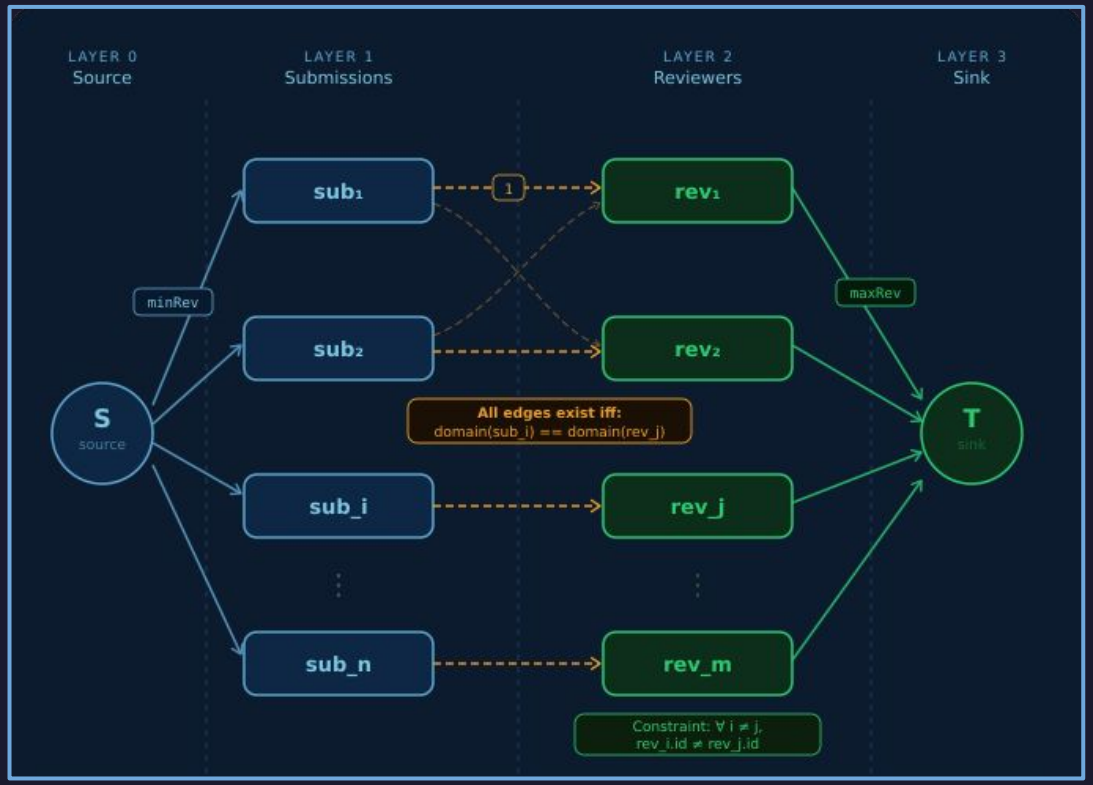
Flow Network — 4-Layer Bipartite Graph

Network Diagram

LEGEND

- ⟶ Unconditional edge (S→sub, rev→T)
- ⇢ Conditional edge — only if domain(sub_i) == domain(rev_j)
- Unique reviewer IDs: ∀ i ≠ j, rev_i.id ≠ rev_j.id

- `minRev` = S → sub capacity
- `maxRev` = rev → T capacity
- `1` = sub → rev capacity



Dinic's Algorithm

1. bfsLevel — O(V+E)

BFS from source to build layered graph.

Each vertex gets a level (distance from source).

Only follows edges with residual capacity > 0.

Returns false when sink is unreachable → done.

2. dfsSend — O(V) per call

DFS through level graph.

Only moves to level + 1 (no sideways/backward).

Current-edge pointer (iter) skips exhausted edges.

Pushes min(residual) along path. Updates flow + reverse.

3. dinic — O(E·√V)

Outer loop:
bfsLevel → rebuild layers

Inner loop:
dfsSend until no more augmenting paths

O(√V) phases on unit-cap bipartite graphs.
Each phase: O(E) total work.

Why Dinic's over Edmonds-Karp? EK: $O(V \cdot E^2)$ vs Dinic's: $O(E \cdot \sqrt{V})$ on unit-capacity bipartite. Current-edge pointer is the key optimization — each edge visited at most once per phase.

Live Demo

[SWITCH TO TERMINAL]

1. Run: ./run.sh
2. Load a dataset file (e.g., dataset5)
3. Show submissions, reviewers, parameters
4. Check file errors (InputValidator)
5. Run assignment ⟶ show results + output file
6. Run all datasets ⟶ batch summary + error report
7. Batch mode: ./run.sh -b input output
8. Error handling: invalid file, missing fields, shifted CSV

Risk Analysis — K = 1

Approach

For each reviewer R_i:

1. Remove R_i from the reviewer list
2. Rebuild entire flow network
3. Run Dinic's algorithm
4. If maxFlow < totalRequired ⟶ R_i is at-risk

Test R independent scenarios, one per reviewer.

Complexity

R iterations × full Dinic's each:

$O(R \cdot E \cdot \sqrt{V})$

(S - submissions, R - Reviewers)
where $E = O(S \cdot R)$ and $V = O(S+R)$

⟶ $O(R \cdot S \cdot R \cdot \sqrt{(S+R)}) = O(S \cdot R^2 \cdot \sqrt{(S+R)})$

Example — dataset9 (domain 3 bottleneck)

S5(domain 3), S6(domain 3) need 2 reviews each ⟶ 4 review slots needed
R7(expertise 3), R8(expertise 3) max 2 reviews each ⟶ 4 slots available (exactly tight!)

Remove R7 ⟶ only R8 remains ⟶ 2 slots for 4 needed ⟶ S5 or S6 can't get enough reviews
Remove R8 ⟶ same problem. Both R7 and R8 are at-risk reviewers.
Remove R1 (expertise 1) ⟶ R2, R3 still cover domain 1 ⟶ no problem.

K > 1 — Non-Trivial Example

Setup: 3 domains, 2 reviewers each

S1(domain 1) S2(domain 2) S3(domain 3)
MinReviews = 1, MaxReviewsPerReviewer = 1

R1(exp 1) R2(exp 1) ← cover S1
R3(exp 2) R4(exp 2) ← cover S2
R5(exp 3) R6(exp 3) ← cover S3

K=1 Result: ALL CLEAR

Remove R1 → R2 still covers S1 ✓
Remove R2 → R1 still covers S1 ✓
Remove R3 → R4 still covers S2 ✓
Remove R4 → R3 still covers S2 ✓
Remove R5 → R6 still covers S3 ✓
Remove R6 → R5 still covers S3 ✓
No single reviewer is at-risk!

K=2 Result: HIDDEN VULNERABILITIES

Remove {R1, R2} → NO reviewer for domain 1 ✗
Remove {R3, R4} → NO reviewer for domain 2 ✗
Remove {R5, R6} → NO reviewer for domain 3 ✗

3 critical pairs that K=1 completely missed!

Remove {R1, R3} → R2 covers S1, R4 covers S2 ✓
Remove {R1, R5} → R2 covers S1, R6 covers S3 ✓
Remove {R2, R4} → R1 covers S1, R3 covers S2 ✓

Cross-domain pairs are safe — each domain retains at least one reviewer.

Risk Analysis — K > 1

The Problem

"If any K reviewers simultaneously fail,
can all submissions still be reviewed?"

Must test every combination of K reviewers
from R total → C(R,K) subsets to evaluate.

Combinatorial Explosion

K=1: C(20,1) = 20 scenarios
K=2: C(20,2) = 190 scenarios
K=3: C(20,3) = 1,140 scenarios
K=5: C(20,5) = 15,504 scenarios
Each scenario = full Dinic's run

Approach 1: Brute Force

For each subset S of K reviewers:
- Remove all in S
- Rebuild network
- Run Dinic's
- Check if fullySatisfied

$O(C(R,K) \cdot E \cdot \sqrt{V})$
Exponential in K

Approach 2: Min-Cut

Max-flow = Min-cut (theorem)

After Dinic's, the last failed
BFS reveals the min-cut.

Reviewers on the cut boundary
are the bottleneck.

$O(E \cdot \sqrt{V})$ — just 1 max-flow run
Finds one critical set.

Approach 3: Iterative

Build on K=1 results:

For each at-risk R_i:
- Remove R_i
- Run K=1 on remaining
- New at-risk → pair found

$O(|A| \cdot R \cdot E \cdot \sqrt{V})$
$|A|$ = K=1 at-risk set size

K > 1 — Min-Cut Theorem Deep Dive

Max-Flow = Min-Cut (Ford-Fulkerson Theorem)

The max-flow value equals the capacity of the minimum cut that separates source from sink.

A cut = partition of vertices into two sets (S-side containing source, T-side containing sink).

The min-cut tells us WHERE the bottleneck is.

Extracting the Min-Cut for Free

After Dinic's, the last bfsLevel fails to reach the sink.

S-side = vertices with level != -1 (reachable)
T-side = vertices with level == -1 (unreachable)

Edges crossing S⟶T with full flow = the min-cut.
No extra computation needed!

Connecting Min-Cut to Risk Analysis

If the min-cut passes through K reviewer⟶sink edges:
Those K reviewers are the bottleneck.
Removing them drops max-flow below required.

If min-cut passes through reviewer⟶sink edges of reviewers {R2, R5, R8}:
⟶ Removing {R2, R5, R8} breaks the assignment.
⟶ They form a critical set of size K=3.

Limitation

Min-cut finds ONE critical set, not ALL.

There may be multiple distinct sets of K reviewers whose removal breaks things.

To find ALL critical sets ⟶ still need brute force O(C(R,K)) enumeration.

Min-cut is best for: "Is there ANY group of K that breaks things?"

K > 1 — When to Use Which Approach

Approach	Complexity	Finds	Best When
Brute Force	$O(C(R,K)\cdot E\cdot\sqrt{V})$	ALL critical sets	Small K (1-3), small R Need exhaustive list
Min-Cut Analysis	$O(E\cdot\sqrt{V})$	ONE critical set	Large K, large R "Does ANY K-set break it?"
Iterative Deepening	$O(\|A\|\cdot R\cdot E\cdot\sqrt{V})$	Critical sets built from K=1 results	K=1 at-risk set is small Good average-case

Pseudocode: Brute Force K>1

```
function riskAnalysisK(subs, revs, params, mode, K):
  atRiskSets = []
  for each subset S of size K from revs:
    reduced = revs \ S          // remove K reviewers
    result = buildAndSolve(subs, reduced, params, mode)
    if NOT result.fullySatisfied:
      atRiskSets.append(S)
  return atRiskSets
```

Key Takeaway

K>1 risk analysis is fundamentally harder than K=1 because of combinatorial explosion.

No known polynomial-time algorithm finds ALL critical K-sets.

In practice: use min-cut for quick check, brute force for small K, iterative for pruning when K=1 results are available.

K > 1 — Expected Output & Reporting

Input: K=2 Example

#Submissions
1, "AI Planning", Alice, a@m.com, 1
2, "Secure DBs", Bob, b@m.com, 2
3, "GPU Scheduling", Carla, c@m.com, 3

#Reviewers
1, John, j@m.com, 1 ← domain 1
2, Sarah, s@m.com, 1 ← domain 1
3, Pedro, p@m.com, 2 ← domain 2
4, Rui, r@m.com, 2 ← domain 2
5, Ana, a@m.com, 3 ← domain 3
6, Luis, l@m.com, 3 ← domain 3

MinReviews=1, MaxReviews=1
RiskAnalysis=2

Output: Risk Analysis K=2

#Risk Analysis: 2
1, 2 ← removing R1+R2 breaks domain 1
3, 4 ← removing R3+R4 breaks domain 2
5, 6 ← removing R5+R6 breaks domain 3

Each line: a pair of reviewers whose simultaneous absence is critical.

What K=2 Reveals That K=1 Misses

K=1 says: "No single reviewer is critical."
K=2 says: "3 pairs are catastrophic."

Conference organizers can now ensure same-domain pairs have backup plans.

Practical use: increase MaxReviewsPerReviewer or recruit extra reviewers in domains with K-vulnerable pairs.

General Formulation — All Domain Modes

Mode 1: Primary only

sub.primary == rev.primary

Sparse graph.
Each submission connects
to ~R/K reviewers.

E = O(S·R/K)

Mode 2: +Sec. Submission

sub.primary == rev.primary
sub.secondary == rev.primary

Submission side opens up.
Papers reach more reviewers
through secondary domains.

E = O(2·S·R/K)

Mode 3: All combinations

4 matching possibilities:
- sub.pri == rev.pri
- sub.sec == rev.pri
- sub.pri == rev.sec
- sub.sec == rev.sec

E = O(4·S·R/K) ⟶ O(S·R)

Impact on Complexity

Same 4-layer graph, same Dinic's algorithm.
Only edge density changes.

Mode 1: $O(S \cdot R/K \cdot \sqrt{(S+R)})$ Mode 3: $O(S \cdot R \cdot \sqrt{(S+R)})$

Key Trade-off

More edges ⟶ more possible assignments
⟶ higher max-flow (better coverage)
⟶ more resilience to reviewer absence
BUT: all matches treated equally (no preference)

Match Quality & Optimality

The Limitation of Plain Max-Flow

Max-flow maximizes total assignments but treats ALL matches equally.

A primary-primary match (perfect fit) has the same weight as a secondary-secondary match (weak fit).

Max-flow finds maximum QUANTITY, not QUALITY.

Solution: Min-Cost Max-Flow

Assign costs to edges by match type:

pri → pri: cost 0 (best)
sec → pri: cost 1
pri → sec: cost 2
sec → sec: cost 3 (weakest)

O(V·E·log V) — more expensive,
but gives qualitatively better results.

The Power of Secondary Domains — Cross-Domain Bridges

Example: Domain 3 has only R7(pri:3) and R8(pri:3, sec:2). In Mode 1, domain 3 is a bottleneck.

In Mode 3: R8 can also review domain 2 papers. Domain 2 reviewers with secondary expertise 3 can help domain 3. Secondary expertise creates 'bridges' between domains — the system degrades gracefully.

Like a train network: Mode 1 has direct routes only. Mode 3 adds connecting routes between cities.

General Formulation — Edge Density Example

dataset3 — Mode 1 (primary only): 9 edges

S1(dom 1,sec 2) ⟶ R1(1), R5(1), R7(1)
S2(dom 2,sec 3) ⟶ R2(2), R6(2), R8(2)
S3(dom 3,sec 4) ⟶ R3(3), R4(3), R9(3)

Each submission sees exactly 3 reviewers.
Domains are isolated silos.
If domain 3 has a shortage ⟶ no escape route.

dataset3 — Mode 3 (all combos): ~24 edges

S1(1,2) ⟶ R1(1,2) R2(2,3) R4(3,1) R5(1,3)
R6(2,4) R7(1,2) R8(2,3) R9(3,1)
S2(2,3) ⟶ R1(1,2) R2(2,3) R3(3,4) R4(3,1)
R5(1,3) R6(2,4) R7(1,2) R8(2,3) R9(3,1)
S3(3,4) ⟶ R2(2,3) R3(3,4) R4(3,1) R5(1,3)
R6(2,4) R8(2,3) R9(3,1)

Each submission sees 7-9 reviewers. ~3x denser!

Mode 1: Isolated Silos

Domain 1: S1 ⟷ {R1,R5,R7}
Domain 2: S2 ⟷ {R2,R6,R8}
Domain 3: S3 ⟷ {R3,R4,R9}

No cross-domain flow possible.

Mode 3: Connected Network

Domains interconnected via secondary expertise bridges.

R4(pri:3,sec:1) bridges dom 1↔3
R2(pri:2,sec:3) bridges dom 2↔3
Flow can reroute around bottlenecks.

Impact on Max-Flow

Mode 1 maxFlow: limited by smallest domain coverage.

Mode 3 maxFlow: ≥ Mode 1.
More paths = more flow.
Strictly better or equal coverage.

The Reviewer Competition Problem

The Scenario

S1(primary: AI) S2(primary: DB, secondary: AI)
R1(primary: AI, maxReviews: 1) R2(primary: DB, maxReviews: 1)
MinReviews = 1

In Mode 3: Both S1 and S2 can reach R1.
S1 ⟶ R1: primary-primary match (AI-AI)
S2 ⟶ R1: secondary-primary match (AI-AI)

But R1 can only do 1 review. Who gets R1?

Why Greedy Assignment Fails

Greedy: assign S2⟶R1 first (secondary match).
Then: S1 has NO reviewer left ⟶ FAIL.

Greedy made a locally reasonable choice that caused a global failure.

Like giving the last hospital bed to a patient who has alternatives, blocking the one who has no other option.

Max-Flow Solves It Optimally

The flow network considers ALL paths:

Source⟶S1⟶R1⟶Sink (only path for S1)
Source⟶S2⟶R1⟶Sink (one option for S2)
Source⟶S2⟶R2⟶Sink (other option for S2)

Dinic's finds: S1⟶R1, S2⟶R2. maxFlow = 2.
Both submissions satisfied. Globally optimal.

Why This Is the Key Insight of T2.4

Secondary domains create MORE edges but also MORE competition for reviewers.

A naive algorithm might assign a reviewer to a weak match, blocking a strong match that has no alternatives.

Max-flow's global optimization is what makes it the RIGHT formulation it considers all constraints simultaneously.

Beyond Max-Flow — Quality-Aware Assignment

Plain Max-Flow Limitation

Max-flow maximizes QUANTITY of assignments.
It does NOT maximize QUALITY of matches.

An AI expert reviewing an AI paper (pri→pri, cost 0)
is treated the same as a DB expert reviewing
an AI paper on their secondary (sec→sec, cost 3).

Min-Cost Max-Flow: Edge Costs

Match Type	Cost	Quality
sub.pri == rev.pri	0	Perfect
sub.sec == rev.pri	1	Good
sub.pri == rev.sec	2	Acceptable
sub.sec == rev.sec	3	Weak

How Min-Cost Max-Flow Works

1. Find max-flow (same total assignments).
2. Among all max-flow solutions, pick the one with minimum total edge cost.
3. Result: same number of reviews, but reviewers are better matched to papers.

Complexity Comparison

Plain Dinic's: $O(E \cdot \sqrt{V})$
Min-Cost Max-Flow: $O(V \cdot E \cdot \log V)$ (using Dijkstra with Johnson)

Significantly more expensive, but
produces qualitatively superior assignments.

Summary: Max-flow answers "Can we assign enough reviews?" Min-cost max-flow answers "Can we assign the BEST reviews?"
Both use the same 4-layer network. The difference is whether edges have only capacity (max-flow) or capacity + cost (min-cost).

Documentation & Complexity Summary

Doxygen Coverage

All public methods documented with @brief, @param, @return.
Time complexity annotations on all relevant algorithms.
Generated HTML + LaTeX in docs/ directory.

Time Complexity Summary

CSVParser::parseFile	O(L)	L = total chars in file
InputValidator::validate	O(S·R)	dominated by domain coverage check
FlowNetwork::bfsLevel	O(V + E)	standard BFS
FlowNetwork::dfsSend	O(E) total	per phase, amortized via current-edge pointer
FlowNetwork::dinic	$O(E \cdot \sqrt{V})$	$\sqrt{V}$ phases × O(E) per phase
FlowNetwork::buildAndSolve	$O(S{\cdot}R \cdot \sqrt{(S+R)})$	graph construction + Dinic's
FlowNetwork::riskAnalysisK1	$O(S{\cdot}R^2 \cdot \sqrt{(S+R)})$	R × full buildAndSolve
OutputWriter::write	O(A log A)	A = number of assignments (sorting)

Thank You

Questions?

Quick Throwback:

Dinic's Max-Flow — $O(E \cdot \sqrt{V})$ on unit-capacity bipartite graphs

14/14 datasets produce correct results

3 domain-matching modes with full validation pipeline

Risk analysis K=1 implemented, K>1 outlined with min-cut approach